

Backpropagation e Treinamento de RNAs

Dataset RAG — Circuitos & Redes Neurais Artificiais

1. Função de Perda

A função de perda mede o erro entre a saída predita e o valor real. A escolha depende do tipo de problema:

MSE (Regressao): $L = (1/n) * \text{soma}((y_i - \hat{y}_i)^2)$

Cross-Entropy (Classificacao binaria): $L = -[y * \log(p) + (1-y) * \log(1-p)]$

Cross-Entropy (Multiclasse): $L = -\text{soma}(y_i * \log(p_i))$

2. Gradiente Descendente

O objetivo do treinamento é minimizar a função de perda ajustando os pesos. O gradiente descendente atualiza os pesos na direção oposta ao gradiente:

$w_{\text{novo}} = w_{\text{antigo}} - \alpha * (dL/dw)$

Onde α é a taxa de aprendizado (learning rate). Um α muito alto causa oscilação; muito baixo causa convergência lenta. Valores típicos: 0.001 a 0.1.

3. Backpropagation

O backpropagation calcula os gradientes de todos os pesos usando a regra da cadeia da derivação composta. O processo tem duas fases:

Forward pass: os dados são propagados da entrada para a saída, calculando as ativações de cada camada e a perda final.

Backward pass: o erro é propagado da saída para a entrada, calculando dL/dw para cada peso via regra da cadeia: $dL/dw = dL/dy * dy/dz * dz/dw$.

4. Variantes do Gradiente Descendente

Batch GD: usa todo o dataset para cada atualização. Preciso mas lento.

SGD (Stochastic): usa um exemplo por vez. Rápido mas ruidoso.

Mini-batch GD: usa lotes de 32-256 exemplos. Melhor equilíbrio.

5. Otimizadores Avancados

Momentum: acumula gradientes passados para acelerar a convergência. $v = \beta * v + (1-\beta) * \text{grad}$; $w = w - \alpha * v$.

Adam (Adaptive Moment Estimation): combina momentum com RMSprop. É o otimizador padrão na maioria das aplicações. Parâmetros típicos: $\alpha=0.001$, $\beta_1=0.9$, $\beta_2=0.999$.

RMSprop: divide o gradiente pela média quadrática dos gradientes recentes, estabilizando o treinamento.